

Open in app ↗

 Medium

★ Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)



Advanced Graphics Programming in Unreal, part 5

17 min read · Jun 25, 2025



William Mishra-Manning



Listen



Share



More

This is a seven-part article series.

Part 1: [Introduction](#)

Part 2: [Rendering Abstractions \(RHI, RDG\) and Threading](#)

Part 3: [Scenes, Views, and SceneViewExtension](#)

Part 4: [Global Shaders](#)

Part 5: Simple Material Shaders

Part 6: [Advanced Material Shaders](#)

Part 7: [Mesh-Material Shaders and Mesh Passes](#)

Basic Material Shaders

This article will explain how to replace Global Shaders, hand-coded from beginning to end by the graphics programmer, with Material Shaders which can inject code from a tech artist's Material graph.

We'll only get Material Shaders working at a basic level, by hacking on existing Material graph nodes. The next article will expand to arbitrary custom input and output nodes.

Theory

All about Materials

What are Materials, really? At first blush people see them as a fancy editor for making shaders. However, this doesn't quite fit — Unreal has dozens of render passes that a surface Material can participate in, each with its own shader; plus who-knows-how-many offscreen shaders under the hood of Lumen and Nanite that are all doing totally different things with your object. Meanwhile Epic reserves the right to add new fancy render passes in the future, with new shaders which your Material must seamlessly integrate with somehow.

The reality is that Materials are one part of the *configuration* of a shader. They configure “how to compute properties” — how to calculate Base Color, Roughness, Metallic, World Position Offset, etc. They also have tons of meta-configuration for the lighting model, blend mode, culling, and so on to inform render passes of how they want to be used. Shaders, on the other hand, are the things which take a Material, execute its relevant output pins to get specific values like Base Color or Opacity, and then calculate a specific output using those values.

Materials themselves do not know or care how to write to the G-buffer, or how motion vectors will be encoded, or how to read from Lumen data structures, or any other input/output stuff. They only know how to describe the thing they're rendering.

So what are Materials really? **They are a common language for the tech-artist to provide code and data to the graphics programmer.**

Speaking the Language

The Material language has multiple “dialects”, in the form of different *Domains*. Each domain provides a different set of output pins and draw settings. Post-Process domain Materials only have a Color pin, while Surface domain Materials have tons of physically-based output pins and lots of settings related to the lighting model.

The graphics programmer picks a target Material domain for their shader, executes whatever Material output pins they care about in that domain, then calculates

whatever final outputs they need for their pass. The Material is none the wiser about how it was used.

You can also extend this language with custom input and output nodes for the Material graph — without changing any engine code! However we will wait to cover this until the next article. For now, we'll settle for the simple yet hacky approach of interpreting standard pins as custom pass outputs.

Why Bother?

The separation between shaders and Materials is hugely advantageous because it decouples fast-iterating tech art work from dense engineer work. Compare it to Unity's URP, where a custom Shaderlab shader needs to provide one pass for every render feature that the engine might ask it to participate in. If an engineer adds a new render feature to the game, the tech artist needs to go back and incorporate that feature's passes into all their existing shaders/Materials.

Back in the days of Unity's legacy BiRP you could write a single "surface shader" which described an object's physical properties and left the hard lighting details to the engine; Unreal's Materials are like surface shaders but far more generalized. You can, for example, build a Compute Shader which takes most of its logic from the Material graph. Shaders become more like a framework for artists to play in than a specific effect.

Implementing Material Shaders

For now we'll focus on screen-space and compute work, because rendering 3D scene geometry requires the much more complex *Mesh-Material Shaders*. The final article in this series will cover those.

Creating a Material Shader is extremely similar to creating a Global Shader. Here is how you can upgrade a Global to Material:

- Make your class inherit from `FMaterialShader` rather than `FGlobalShader`, and update the various macro invocations accordingly.
- Change `SHADER_USE_PARAMETER_STRUCT(FMyShader, FGlobalShader)` to `SHADER_USE_PARAMETER_STRUCT_WITH_LEGACY_BASE(FMyShader, FMaterialShader)`. This

tells Unreal not to worry when some shader parameters are missing from your parameter struct (because the Material instance has parameters of its own). If you don't do this then you get a harmless-but-annoying assert failure whenever the shader is compiled.

- Replace all calls to the global `SetShaderParameters(...)` with some new global helper functions, `SetShaderParametersMixed[VS|CS|PS|etc](...)`. This new helper will call the original *and* call your shader's inherited member function `SetParameters(...)`, which uploads all the Material parameters. If you don't do this then your user's Materials will see all their params set to default/garbage values!
- If you followed the previous article about Global Shaders, then you never called `SetShaderParameters(...)` manually in the first place. Most draw operations can be done by calling Unreal's helper functions, and nearly all those helper functions conveniently call `SetShaderParameters(...)` for you. Unfortunately, now that you are replacing those calls with `SetShaderParametersMixed[X](...)`, you need to change how you use these helper functions. Look for new overloads which take a "setup lambda". This is where they previously called `SetShaderParameters(...)`, and where you are meant to call `SetShaderParametersMixed[X](...)`.
- Instead of extracting your global shader from the global shader-map, extract your Material shader from a specific Material's shader-map. There is a complicated snippet to extract Material shaders, which my library simplifies into `EGP::FindMaterialShaders_RenderThread(...)`.
- Inject each Material's code into your shader file with `#include "/Engine/Generated/Material.ush"`. To help your IDE see some of the code that Unreal generates for Materials, I highly recommend this intellisense trick which works in both Visual Studio and JetBrains Rider:

```
#if defined(__INTELLISENSE__) || defined(__RESHARPER__)
    //The Material template file is malformed
    // but IDE's can parse it well enough!
    #include "/Engine/Private/MaterialTemplate.ush"
#else
    //In the real shader compiler, use the real Material code.
```

```
#include "/Engine/Generated/Material.ush"  
#endif
```

***NOTE:** This include statement is where you can inject new input nodes into Material graphs. We will cover the technique in the next article but the gist is to define some static globals right before the `#include`, write to those globals before invoking Material code, then create a Material Function asset which reads from those globals using a Custom HLSL node.*

***UPDATE:** I just learned something important that cost me a lot of time at work! I already mentioned that new shaders have to be defined in a module loaded on `PostConfigInit`. If you fail to respect this, Unreal normally logs an error on startup...but only for Global Shaders. If you load a Material Shader at the wrong phase, then it silently propagates until you go to run a standalone build and get sporadic crashes about missing shader-maps.*

Dispatching Material Shaders

The process of invoking Material code in a shader varies a lot by the shader's frequency (vertex/pixel/compute), the Material domain, and especially whether it's a Mesh-Material shader.

For non-3D work I recommend setting your Material shaders up like the engine's post-process shader, operating on materials in the Post-Process domain. This domain comes with helpful infrastructure, like special input nodes that expose multiple textures.

It requires a lot of boilerplate to set up, but my library handles all of it for you within the header `EGP_PostProcessMaterialShaders.h`. The rest of this article assumes you are using that approach (and see the next section for the HLSL side of it). If you want to go your own way, at least read through EGP's relevant headers and shaders to understand what it's doing first!

EGP offers two kinds of shader passes for post-process Materials: off-screen compute work, called a *Simulation* pass, and screen-space compute/pixel work, called a *Screen-Space* pass. The difference is that Simulation passes run in a vacuum, without Viewport data. To execute a shader using either of these pass types, do the

following:

1. Have your shader inherit from one of our custom `FMaterialShader` subtypes,

`EGP::FSimulationShader` OR `EGP::FScreenSpaceShader` .

2. Include an extra line in your parameter struct, either

`EGP_SIMULATION_PASS_MATERIAL_DATA()` OR `EGP_SCREEN_SPACE_PASS_MATERIAL_DATA()` .

3. Define the inputs to the post-process Material graph by creating an instance of an “input” struct, either `EGP::FSimulationPassMaterialInputs` OR

`EGP::FScreenSpacePassMaterialInputs` .

4. Tell my library how to execute your pass by creating an instance of a “state” struct. Pick from one of the following types:

–For **simulation passes**, use `EGP::FSimulationPassState` . Work-group count can be given immediately, or deferred to a render-thread lambda executed right before dispatching the shader, or pulled from a GPU buffer (a.k.a. “indirect dispatch”).

–For **screen-space passes that use a vertex+pixel shader**, use

`EGP::FScreenSpacePassRenderState` .

–For **screen-space passes that use a compute shader**, use

`EGP::FScreenSpacePassComputeState` . Like with simulation passes, work-group count can be given in several ways.

5. Call a function to queue up the pass, and provide all this data to it. Either

`EGP::AddSimulationMaterialPass(...)` , `EGP::AddScreenSpaceRenderPass(...)` , OR

`EGP::AddScreenSpaceComputePass(...)` .

Screen-space Render passes, which use a Vertex and Pixel shader, commonly need a different parameter struct for each shader. If you’re implementing both shaders yourself then make an overarching parameter struct which holds both structs nested inside it, and pass all three into the dispatch call. However in most cases you can use the built-in vertex shader `EGP::FScreenSpaceRenderVS` , and then your pixel shader’s param struct **already is** the overarching one. Extract the vertex shader params with `&pixelParams->ScreenSpacePassData` .

NOTE: Simulation passes conceptually do not have a viewport, but Unreal sadly requires that Material shaders have a view instance when setting up their parameters. So for now you must pass an `FViewInfo` into `AddSimulationMaterialPass(...)`, which means there's not much value to using Simulation passes over Screen-Space. Fixing this may require copying a bit of engine code and ripping out the viewport parts.

Material Shader HLSL

The way you actually execute a Material in HLSL varies based on many details. In any case, following my post-process Material workflow, you should put the following near the top of your shader:

```
#include "EGP/ScreenPass/pre.ush"
// (define any custom Material inputs here)
#include "EGP/ScreenPass/post.ush"
```

This will bring in all the post-process boilerplate and generated Material code.

The start of your main function will depend on the kind of shader you are writing. The ultimate goal is to fill in a “material inputs” struct and a “material parameters” struct; then you can call the Material’s generated functions to compute whatever output pins you want.

For a **Simulation Compute** shader, you have to provide a lot of the standard material inputs by hand:

```
//Fabricate info that is normally computed from the viewport and VS.
uint2 viewportPixel = threadIdx.xy;
float2 uv = (float2(viewportPixel.xy) + 0.5) / float2(MyViewportSize);
float3 reportedWorldPos = float3((uv * 2) - 1, 0);
float reportedDepth = 0;

//Generate the Material structs.
FPixelMaterialInputs mInputs;
FMaterialPixelParameters mParams;
```

```
ScreenPassSetupCS(viewportPixel, uv, reportedWorldPos, reportedDepth,
                  mInputs, mParams);
//If you have custom material inputs, set them up here.

//Now we can invoke material code!
```

For a **Screen-Pass Compute** shader, things are simpler because you can lean on the viewport data:

```
uint2 pixel = threadIdx.xy;
FPixelMaterialInputs mInputs;
FPixelMaterialParameters mParams;
ScreenPassSetupCS(pixel, mInputs, mParams);
//If you have custom material inputs, set them up here.

//Now we can invoke material code!
```

For a **Screen-Pass Vertex** shader, you should probably start by copying the contents of *ExtendedGraphicsProgramming/Shaders/ScreenPass/simple_vs.usf*. Material vertex shaders are pretty simple, and the only struct that needs to get filled in is

FMaterialVertexParameters :

```
float2 uv = ...; //Probably coming from some vertex input
FTexCoordsVSToPS materialInterpolants; //If your PS is also MaterialShader
// then this should be a VS output

FMaterialVertexParameters mParams;
ScreenPassSetupVS(uv, mParams, materialInterpolants);
//If you have custom material inputs, set them up here.

//Now we can invoke material code!
```

Finally, for a **Screen-Pass Pixel** shader you need those VS outputs to initialize the Material:

```
in float4 svPos : SV_Position //Important builtin input
in FTexCoordsVSToPS materialInputsFromVS //Output of our Material VS

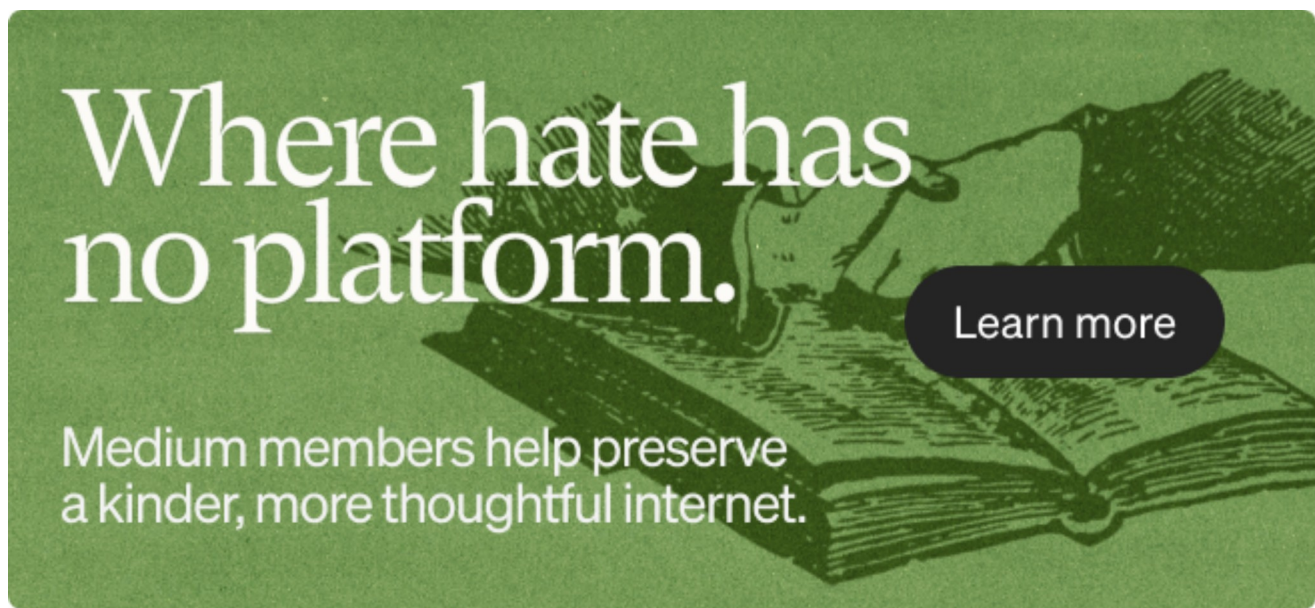
FPixelMaterialInputs matInputs;
FMaterialPixelParameters matParams;
ScreenPassSetupPS(svPos, materialInputsFromVS,
                  matInputs, matParams);
//If you have custom material inputs, set them up here.

//Now we can invoke material code!
```

For a reference of what Material functions and data you can play with, check out the template file that's used to generate them: */Engine/Private/MaterialTemplate.ush*. For example, in a pixel or compute shader you can get the Emissive Color pin by calling `GetMaterialEmissive(matInputs)` . Once we make custom output pins in the next article, you can check for those in a similar way.

Demo

We will convert 3 of our 4 shaders to Material shaders. For variety, we will also change our Simulate shader to be a Simulation compute pass instead of a pixel shader. It makes more sense as a Screen-Space compute pass since it has an associated viewport, but I want to show how to create a full variety of Material shader passes.



NOTE: When using DX12, I get a harmless warning popup that says some shader parameters of Simulate.usf are unbound. This doesn't stop the effect from working, and I don't see the warning with the Vulkan RHI, so I didn't bother tracking down why this happens. But still you may want to switch to Vulkan for now, or remake the Simulate shader as a Screen-Space pass rather than a Simulation pass.

Pass Object takes a Material

To start, remove all previous parameters from our Pass object (initial noise seed and scale; simulation speed and acceleration). In place of that, add a new Material field and a render-thread copy of it.

```
//Inside U_GoL_RenderPass:
public:
    UPROPERTY(BlueprintReadWrite, EditAnywhere)
    UMaterialInterface* EffectMaterial = nullptr;
private:
    // ReSharper disable once CppUE4ProbableMemoryIssuesWithUObject
    UMaterialInterface* effectMaterial_RenderThread = nullptr;
public:
    UMaterialInterface* GetEffectMaterial_RenderThread() const {
        check(IsInRenderingThread());
        return effectMaterial_RenderThread;
    }

//Inside U_GoL_RenderPass::Tick_GameThread(...):
```

```

//Update the render-thread copies of our parameters.
auto* matIn = EffectMaterial;
auto* matOut = &effectMaterial_RenderThread;
ENQUEUE_RENDER_COMMAND(UpdateGoLParams)(
    [matIn, matOut](FRHICommandList& cmds) {
        *matOut = matIn;
    });

```

The last step is to feed a Material to this pass at runtime, but first we need some shaders for the Material to control!

Initialization Shader

Initialization is the simplest Material shader. Previously it generated discrete and continuous starting values using hard-coded noise, but now the Material Graph will feed initial values through the Emissive output's Red and Green channels.

Update the shader file:

```

//Init.usf:
#include "/EGP/ScreenPass/pre.ush"

//Hint to the Materials which shader we're running in.
#define GOL_PASS_INIT 1

#include "/EGP/ScreenPass/post.ush"

//To help intellisense see Material code,
// use this snippet.
#if defined(__INTELLISENSE__) || defined(__RESHARPER__)
#include "/Engine/Private/MaterialTemplate.ush"
#endif

//We will use the standard VS for screen-space passes,
// FScreenSpaceRenderVS.
//Our inputs come from there.
void Main(in FTexCoordsVSToPS materialInterpolants,
          in float4 svPos : SV_Position,
          out float2 newState : SV_Target0)
{
    //Set up the Material.
    FPixelMaterialInputs matInputs;
    FMaterialPixelParameters matParams;

```

```

    ScreenPassSetupPS(svPos, materialInterpolants,
                      matInputs, matParams);

    //Compute the Material's Emissive Color pin
    //    and interpret it as our initial cell state.
    newState.xy = GetMaterialEmissive(matInputs).rg;

    //The first channel is supposed to be a discrete bool.
    newState.x = step(0.5, newState.x);
}

```

Update the shader class:

```

struct FGoLInitializePS : public EGP::FScreenSpaceShader
{
    DECLARE_EXPORTED_SHADER_TYPE(FGoLInitializePS, Material, )

    BEGIN_SHADER_PARAMETER_STRUCT(FParameters, )
        EGP_SCREEN_SPACE_PASS_MATERIAL_DATA()
        RENDER_TARGET_BINDING_SLOTS()
    END_SHADER_PARAMETER_STRUCT()
    SHADER_USE_PARAMETER_STRUCT_WITH_LEGACY_BASE(FGoLInitializePS,
                                                  EGP::FScreenSpaceShader)
};

IMPLEMENT_MATERIAL_SHADER_TYPE(, FGoLInitializePS,
                               TEXT("/GameOfLife/Init.usf"),
                               TEXT("Main"), SF_Pixel);

```

And finally update the `FGameOfLifeView` constructor which invokes it:

```

//The constructor's new arguments:
FGameOfLifeView(FRDGBuilder& graph,
                const FViewInfo& view, const FIntRect& viewportSubset,
                const UMaterialInterface* effectMaterial,
                const FSceneTextureShaderParameters& sceneTextures);

...

```

```

//At the end of the constructor:

//Parameters for initialization
auto* initShaderParams = graph.AllocParameters<FGOLInitializePS::FParameters>()
initShaderParams->RenderTargets[0] = {
    simStateRDG,
    ERenderTargetLoadAction::ENoAction
};
EGP::FScreenSpacePassMaterialInputs postProcessMaterialInputs;
postProcessMaterialInputs.SceneTextures = sceneTextures;
postProcessMaterialInputs.TargetView = &view;
postProcessMaterialInputs.OutputViewportData = FScreenPassTextureViewport{
    simStateRDG
};
postProcessMaterialInputs.InputViewportData = FScreenPassTextureViewport{
    view.ViewRect
};

//Default to opaque blending and no depth/stencil usage
EGP::FScreenSpacePassRenderState renderState{ };

//Dispatch the Material shader
EGP::AddScreenSpaceRenderPass<EGP::FScreenSpaceRenderVS, FGOLInitializePS>(
    graph, RDG_EVENT_NAME("GoL_Initialize"),
    postProcessMaterialInputs, renderState,
    initShaderParams, effectMaterial,
    //Extract the specific VS and PS param structs
    &initShaderParams->ScreenSpacePassData, initShaderParams
);

...

//In F_GOL_PassSVE::PrePostProcessPass_RenderThread(),
//    update our view-data getter:
auto& viewData = Pass->PerViewData.DataForView(
    graph, view,
    //The below arguments get passed to constructor if this is a new view
    Pass->GetEffectMaterial_RenderThread(),
    GetSceneTextureShaderParameters(inputs.SceneTextures)
);

```

Now the Material we give to the pass object in-game can control initialization!

Before creating a Material though, we'll add the other two shaders.

Display Shader

The process to convert this shader is very similar. Rather than computing the final color ourselves, we simply return the Material's emissive color pin. We also need to provide the Material graph with the current sim state, so they can sample from it.

Update the shader code:

```
//Display.usf

#include "/EGP/ScreenPass/pre.usf"
//Hint to Materials which shader we're running in.
#define GOL_PASS_DISPLAY 1
#include "/EGP/ScreenPass/post.usf"

float4 Main(in FTexCoordsVSToPS materialInterpolants,
            in float4 svPos : SV_Position) : SV_Target0
{
    //Set up the Material.
    FPixelMaterialInputs matInputs;
    FMaterialPixelParameters matParams;
    ScreenPassSetupPS(svPos, materialInterpolants, matInputs, matParams);

    //Let the Material decide how to sample and display the effect.
    return float4(GetMaterialEmissive(matInputs), 1.0);
}
```

Conver the shader class:

```
struct FGoLDisplayPS : public EGP::FScreenSpaceShader
{
    DECLARE_EXPORTED_SHADER_TYPE(FGoLDisplayPS, Material, )

    BEGIN_SHADER_PARAMETER_STRUCT(FParameters, )
        EGP_SCREEN_SPACE_PASS_MATERIAL_DATA()
        RENDER_TARGET_BINDING_SLOTS()
    END_SHADER_PARAMETER_STRUCT()
    SHADER_USE_PARAMETER_STRUCT_WITH_LEGACY_BASE(FGoLDisplayPS,
                                                EGP::FScreenSpaceShader)
};
```

```
IMPLEMENT_MATERIAL_SHADER_TYPE(, FGoLDisplayPS,  
    TEXT("/GameOfLife/Display.usf"),  
    TEXT("Main"), SF_Pixel);
```

Update our helper function which dispatches the shader:

```

static void RenderGoLState(FRDGBuilder& graph, const FViewInfo& view,
                          FRDGTextureRef simStateTex,
                          FRHIBlendState* blending,
                          const FRenderTargetBinding& output,
                          const UMaterialInterface* material,
                          const FSceneTextureShaderParameters& sceneTextures)
{
    auto* params = graph.AllocParameters<FGoLDisplayPS::FParameters>();
    params->RenderTargets[0] = output;

    //Provide the "sim state" texture as Post-Process Texture slot 0.
    EGP::FScreenSpacePassMaterialInputs inputs;
    inputs.Textures[0] = GetScreenPassTextureInput(
        FScreenPassTexture{ simStateTex },
        TStaticSamplerState<SF_Bilinear, AM_Clamp, AM_Clamp>::GetRHI()
    );
    //Provide other info about the viewport being drawn to.
    inputs.SceneTextures = sceneTextures;
    inputs.TargetView = &view;
    inputs.InputViewportData = FScreenPassTextureViewport{
        inputs.Textures[0].Texture
    };
    inputs.OutputViewportData = FScreenPassTextureViewport{
        output.GetTexture(), view.ViewRect
    };

    //Configure the PSO for this pass.
    EGP::FScreenSpacePassRenderState state{ blending };

    //Dispatch.
    EGP::AddScreenSpaceRenderPass<EGP::FScreenSpaceRenderVS,
                                  FGoLDisplayPS>(
        graph, RDG_EVENT_NAME("GoL_Display"),
        inputs, state, params, material,
        //Extract the specific VS and PS param structs:
        &params->ScreenSpacePassData, params
    );
}

```

Simulation Shader

This one will have more significant changes. Currently it runs in the pass object's render-thread Tick, but for technical reasons we need View data when dispatching a

Simulation material pass. We'll move that code to the SVE, and the Tick function just accumulates delta-time in between render events.

Add a delta-time counter to the per-view data.

```
//In FGameOfLifeView
float NextTickTime = 0;
```

Update the old tick logic to increment this counter instead of running the Simulate shader.

```
//In U_GOL_RenderPass::Tick_RenderThread:
PerViewData.ForEachView([&](int viewID,
                                FGameOfLifeView& view,
                                ERHIFeatureLevel::Type featureLevel) {
    view.NextTickTime += gameThreadDeltaSeconds;
});
```

Update the shader code to be a Simulation Compute shader rather than a global pixel shader:

```
//Simulate.usf

#include "/EGP/ScreenPass/pre.ush"
//Hint to Materials which shader we're running in.
#define GOL_PASS_SIMULATE 1
#include "/EGP/ScreenPass/post.ush"

float DeltaSeconds;
RWTexture2D<float2> NextSimStateTex;

//The input state texture is part of the Material configuration now;
// it isn't declared ourselves.
#define SimStateTex PostProcessInput_0_Texture
```

```

//The shader class will tell us what group size to use.
[numthreads(SIM_GROUP_SIZE_X, SIM_GROUP_SIZE_Y, SIM_GROUP_SIZE_Z)]
void Main(uint3 threadIdx : SV_DispatchThreadID)
{
    uint2 resolution;
    SimStateTex.GetDimensions(resolution.x, resolution.y);

    uint2 pixel = threadIdx.xy;
    float2 uv = (float2(pixel) + 0.5) / float2(resolution);
    if (any(pixel >= resolution))
        return;

    //Set up the Material.
    FPixelMaterialInputs matInputs;
    FMaterialPixelParameters matParams;
    float3 fakeWorldPos = float3((uv * 2) - 1, 0);
    float fakeFragmentDepth = 0.0;
    ScreenPassSetupCS(pixel, uv, fakeWorldPos, fakeFragmentDepth,
                      matInputs, matParams);

    //Query the GoL rules from the Material,
    //    interpreting its three Emissive outputs
    //    as various living/dying thresholds.
    float3 matOutputs = GetMaterialEmissive(matInputs);
    const float ThresholdTooFew = matOutputs.x,
                ThresholdResurrect = matOutputs.y,
                ThresholdTooMany = matOutputs.z;
    const float OverallSpeed = 0.3;

    // (the core GoL logic is mostly unchanged and omitted here)

    NextSimStateTex[pixel] = float2(targetValue, smoothValue);
}

```

Update the shader's class accordingly.

```

struct FGoLSimulateCS : public EGP::FSimulationShader
{
    DECLARE_EXPORTED_SHADER_TYPE(FGoLSimulateCS, Material, );
    static FIntVector3 GroupSize() { return { 8, 8, 1 }; }

    BEGIN_SHADER_PARAMETER_STRUCT(FParameters, )
        SHADER_PARAMETER(float, DeltaSeconds)
        SHADER_PARAMETER_RDG_TEXTURE_UAV(RWTexture2D<float2>,

```

```

        NextSimStateTex)

    EGP_SIMULATION_PASS_MATERIAL_DATA()
END_SHADER_PARAMETER_STRUCT()
SHADER_USE_PARAMETER_STRUCT_WITH_LEGACY_BASE(FGoLSimulateCS,
                                              EGP::FSimulationShader)

//Feed the group size to the shader as a pre-processor define.
//That way it isn't hard-coded in two places.
static void ModifyCompilationEnvironment(
    const FMaterialShaderPermutationParameters& params,
    FShaderCompilerEnvironment& env
)
{
    EGP::FSimulationShader::ModifyCompilationEnvironment(params, env);
    env.SetDefine(TEXT("SIM_GROUP_SIZE_X"), GroupSize().X);
    env.SetDefine(TEXT("SIM_GROUP_SIZE_Y"), GroupSize().Y);
    env.SetDefine(TEXT("SIM_GROUP_SIZE_Z"), GroupSize().Z);
}
};

IMPLEMENT_MATERIAL_SHADER_TYPE(, FGoLSimulateCS,
    TEXT("/GameOfLife/Simulate.usf"),
    TEXT("Main"),
    SF_Compute);

```

Now rewrite the helper function that dispatches this shader.

```

static void UpdateGoLState(FRDGBuilder& graph, const FViewInfo& view,
    FRDGTextureRef currentSimState,
    FRDGTextureRef nextSimState,
    float deltaSeconds,
    const UMaterialInterface* uMaterial)
{
    check(currentSimState->Desc.Extent == nextSimState->Desc.Extent);

    //Provide the previous state to the material graph
    //    as Post-Process Texture 0.
    EGP::FSimulationPassMaterialInputs inputs;
    inputs.Textures[0] = GetScreenPassTextureInput(
        FScreenPassTexture{ currentSimState },
        TStaticSamplerState<SF_Bilinear, AM_Clamp, AM_Clamp>::GetRHI()
    );

    //Set up our non-Material shader params.

```

```

auto* params = graph.AllocParameters<FGoLSimulateCS::FParameters>();
params->DeltaSeconds = deltaSeconds;
params->NextSimStateTex = graph.CreateUAV(nextSimState);

//Calculate the group count.
EGP::FSimulationPassState state;
state.GroupCount.Set<FIntVector3>(
    FComputeShaderUtils::GetGroupCount(
        FIntVector3{ currentSimState->Desc.Extent.X,
                    currentSimState->Desc.Extent.Y,
                    1 },
        FGoLSimulateCS::GroupSize()
    )
);

//Dispatch.
EGP::AddSimulationMaterialPass<FGoLSimulateCS>(
    graph, RDG_EVENT_NAME("GoL_Tick"),
    inputs, state, view,
    params, uMaterial
);
}

```

Finally, in the SVE's post-process callback, run a Simulate pass before Display.

```

//For any time has passed on the game thread, tick this viewport's sim.
if (viewData.NextTickTime > 0)
{
    auto nextSimStateRDG = RegisterExternalTexture(
        graph, viewData.SimBuffer,
        TEXT("GoL_NextState")
    );
    UpdateGoLState(
        graph, view,
        simStateRDG, nextSimStateRDG,
        viewData.NextTickTime, passMaterial
    );
    viewData.NextTickTime = 0;

    std::swap(viewData.SimBuffer, viewData.SimState);
    simStateRDG = nextSimStateRDG;
}

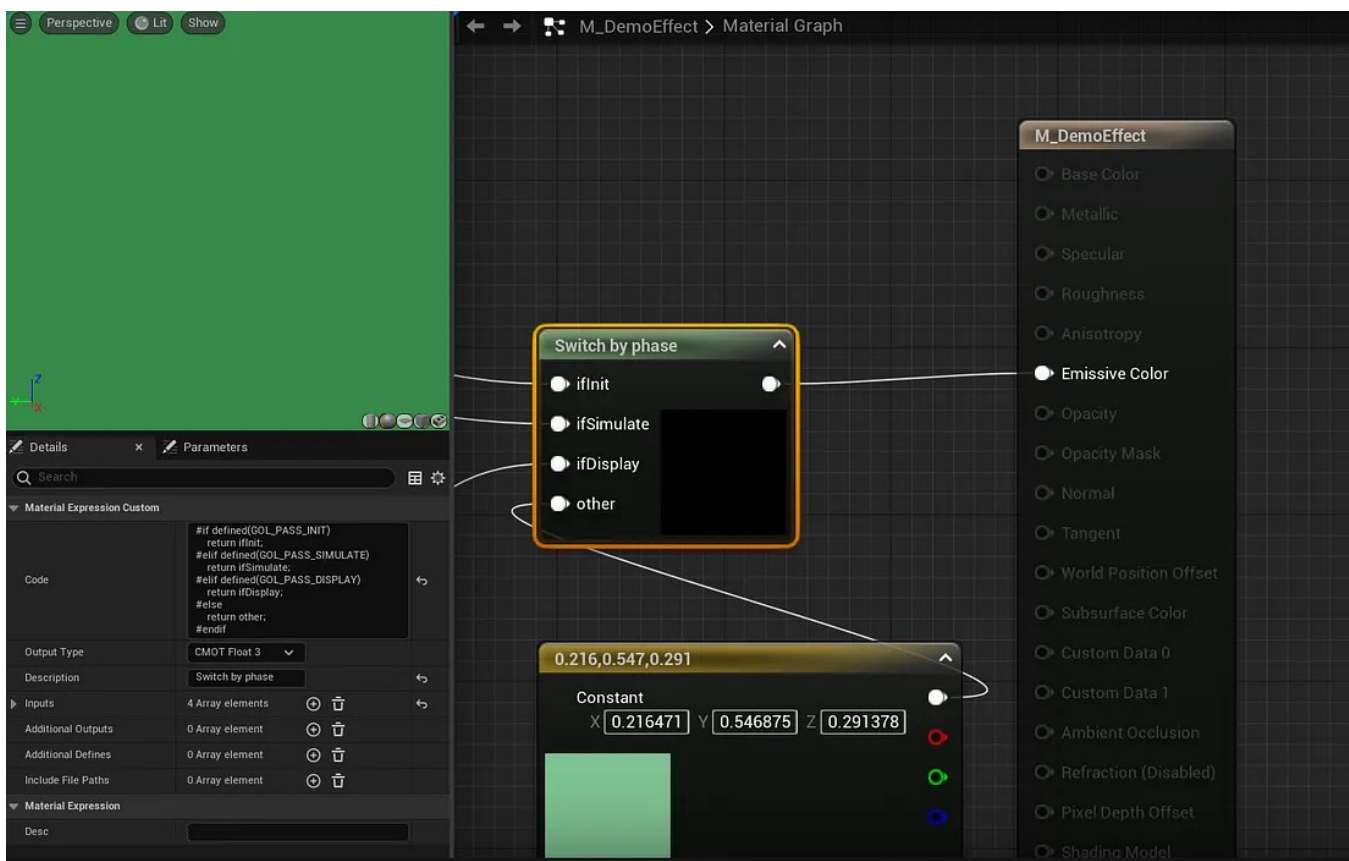
```

Create a pass Material

Our code is fully converted, so let's make a Material to drive the effect.

NOTE: if the Material graph looks janky to you, that's because it is! The next article will greatly improve this workflow.

1. Open up the Unreal editor for the project and create a new Material. Set the domain type to Post-Process.
2. If you recall, each Material Shader defined a preprocessor token to tell us what pass is running — `GOL_PASS_INIT`, `GOL_PASS_SIMULATE`, and `GOL_PASS_DISPLAY`. We can take advantage of this in a Custom HLSL node to switch between different logic based on the pass we're running:



Switching logic by shader stage

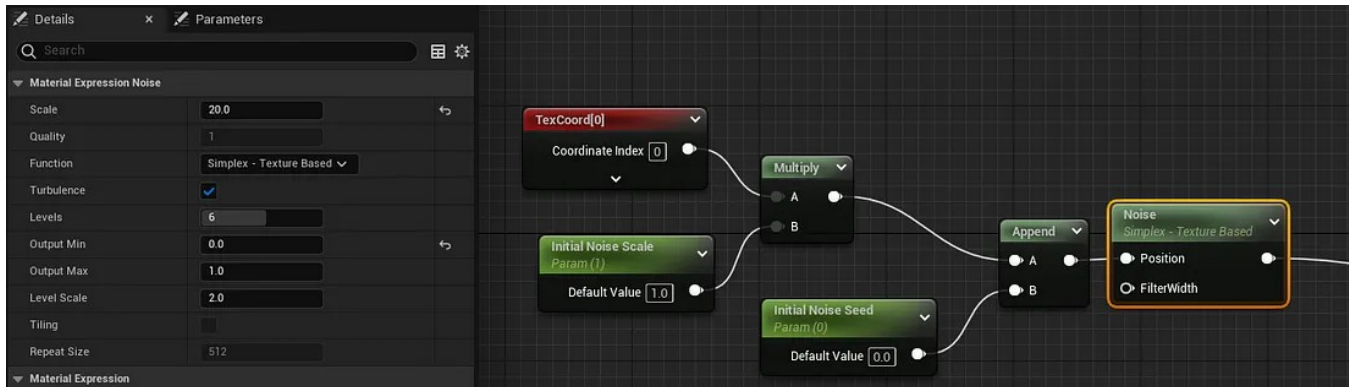
```
//Source for the custom node:
#if defined(GOL_PASS_INIT)
```

```

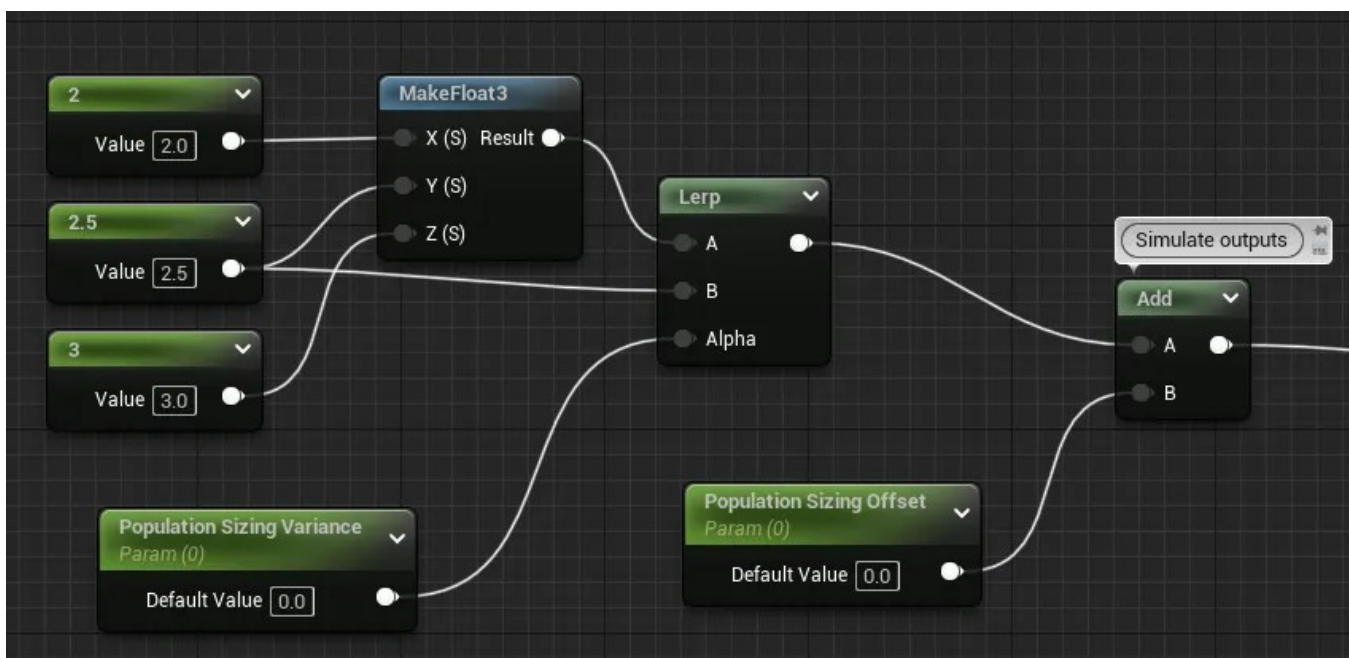
    return ifInit;
#elif defined(GOL_PASS_SIMULATE)
    return ifSimulate;
#elif defined(GOL_PASS_DISPLAY)
    return ifDisplay;
#else
    //Material preview and other contexts
    return other;
#endif

```

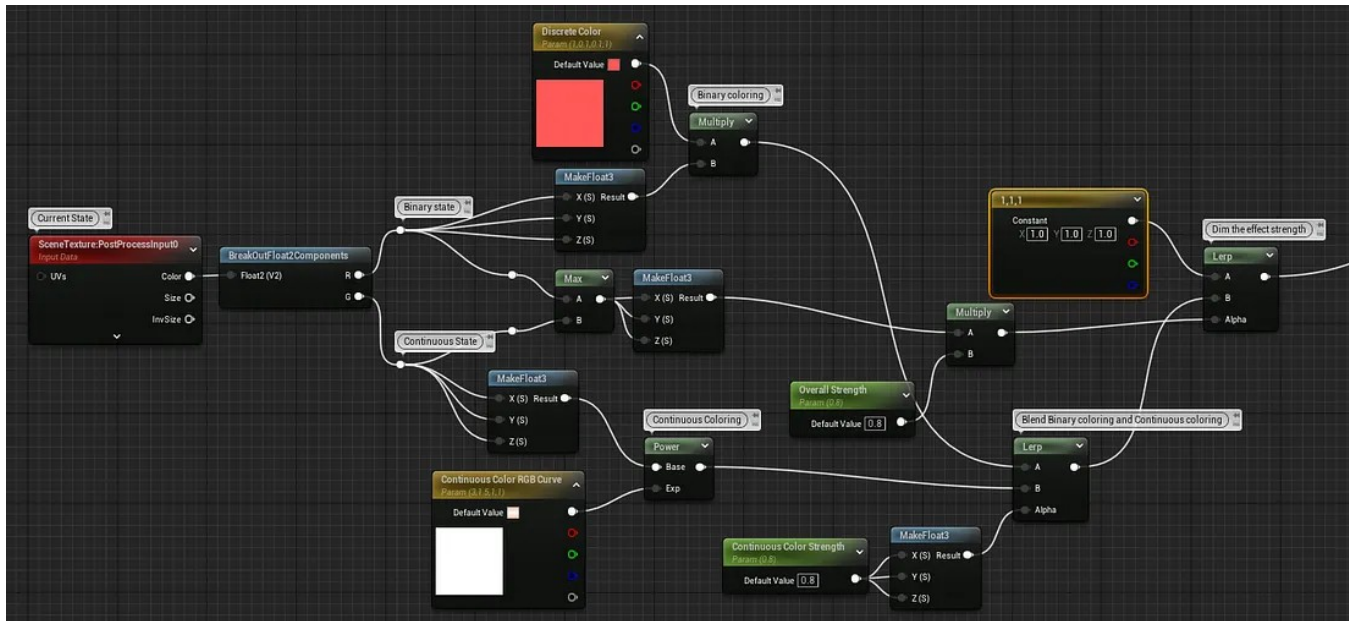
3. Create logic for each of the phases and plug them into the custom Switch node. The below graph recreates the original Global shader logic, with customizable parameters.



Initialize stage logic

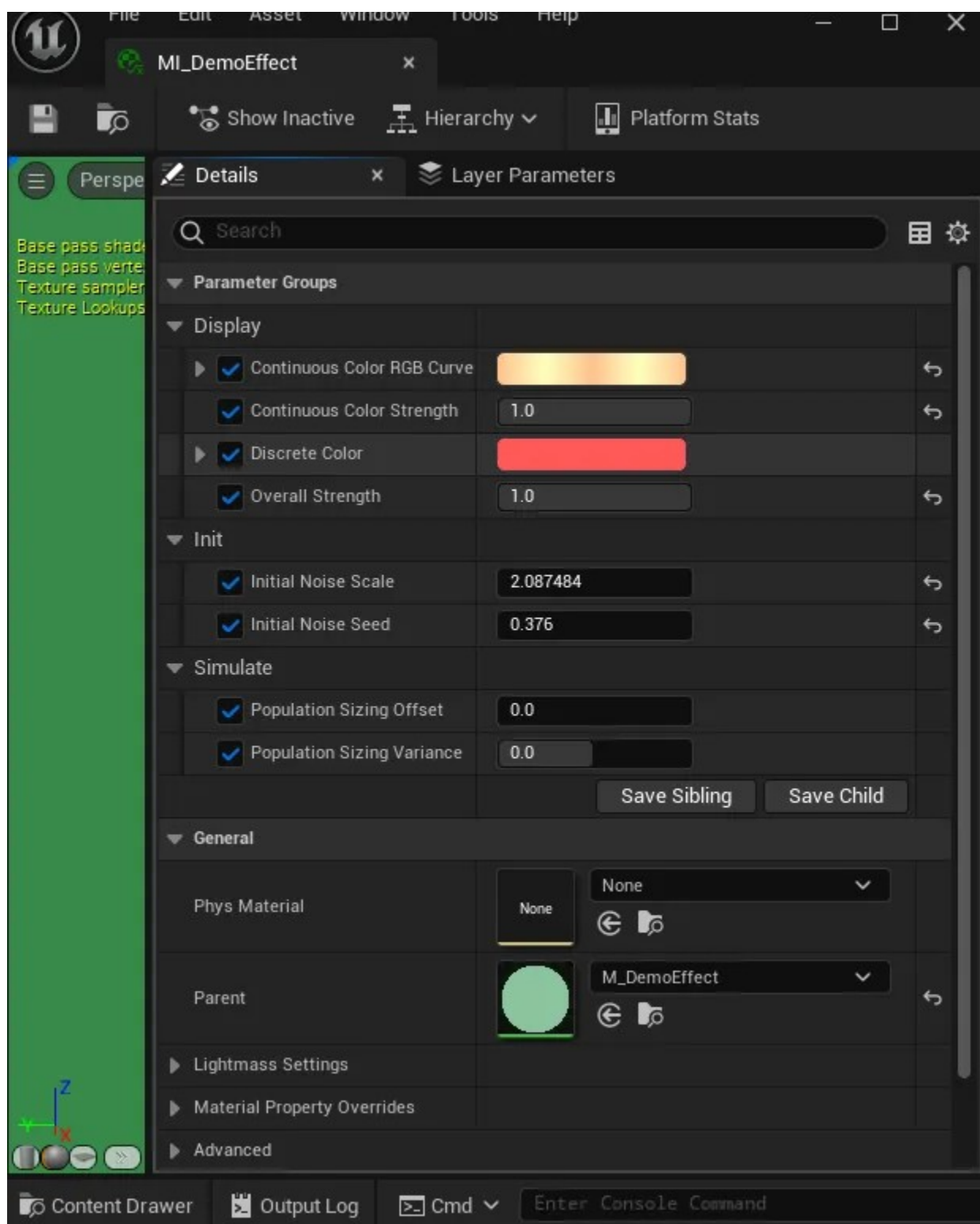


Simulate stage logic



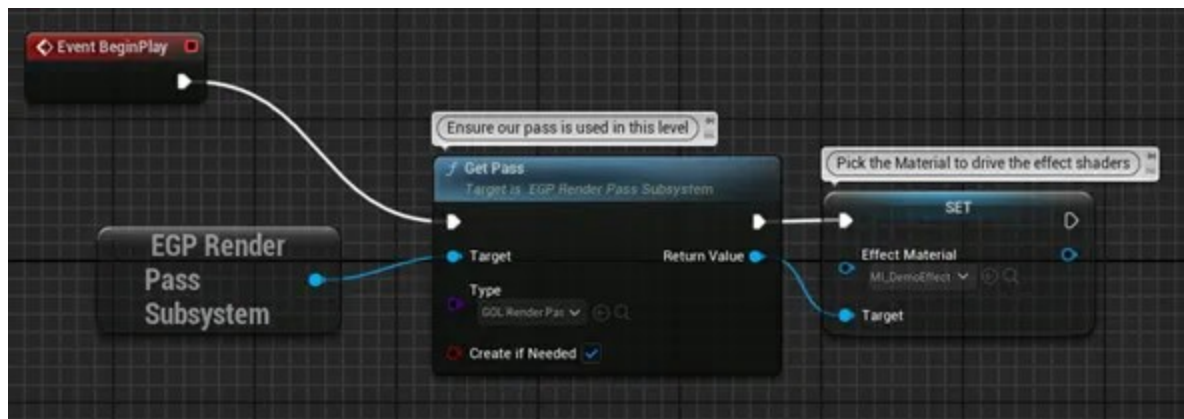
Display stage logic

4. For easier tweaking, create a Material Instance from your Material.



Material Instance that recreates the Global shader logic

5. In the demo scene, set up the render pass to use the Material instance. As mentioned before, I do this setup in the level blueprint.



The level blueprint in my demo scene

And that's everything! Now you should be able to play with that Material instance, or underlying Material, and see immediate changes to the simulation — without even having to restart the game.

Feel free to play around with your new Material, but in the next article we'll improve the UX a lot.

Small Bug Workaround

One problem with this demo is that the Material shaders, including *Initialize*, take a few frames to compile against the Material the first time they're used. In the editor it's no problem after your first run, but in standalone it can mean the initialize shader is executed with the Default Material, creating an empty initial state! To fix this I added the ability to re-run the Initialize shader at will, and had the demo's Game Mode re-run initialization after a few seconds.

I'm sure there must be some elegant way to pre-compile the Material shaders, or even force the render code to wait for compilation before running *Initialize*, but I currently don't know how, so this is good enough for demo purposes. If you really want to see how I implemented this check out the custom Game Mode and Player Controller blueprints, and the C++ function

```
U_GOL_RenderPass::ReInitializeAllViews() .
```

It's possible this could be fixed with PSO precaching, but I haven't looked into it.

Code

The Material shaders are added to the demo in the *part/5* branch:

[heyx3/UnrealExtendedGraphicsProgrammingDemo at part/5](#)

Details

`EGP::FindMaterialShaders_RenderThread(...)` will go through fallback Materials in case the one you provided doesn't work (e.g. it doesn't compile). In the worst-case it falls through to the universal Default Material for the domain you were using, which from my understanding is one with all settings left at defaults and all output pins not written to.

If you want to understand more about dispatching Material shaders, take a look under the hood of `EGP::AddScreenSpaceRenderPass(...)` and *Shaders/ScreenPass/pre.ush*. I like to think my code is pretty well-commented!

If you want to understand more about how Materials are invoked in HLSL, read *Shaders/ScreenPass/post.ush*.

[Unreal Engine 5](#)[Graphics Programming](#)[Shaders](#)[Technical Art](#)[Edit profile](#)

Written by William Mishra-Manning

59 followers · 5 following

Graphics Programming, Procedural Generation

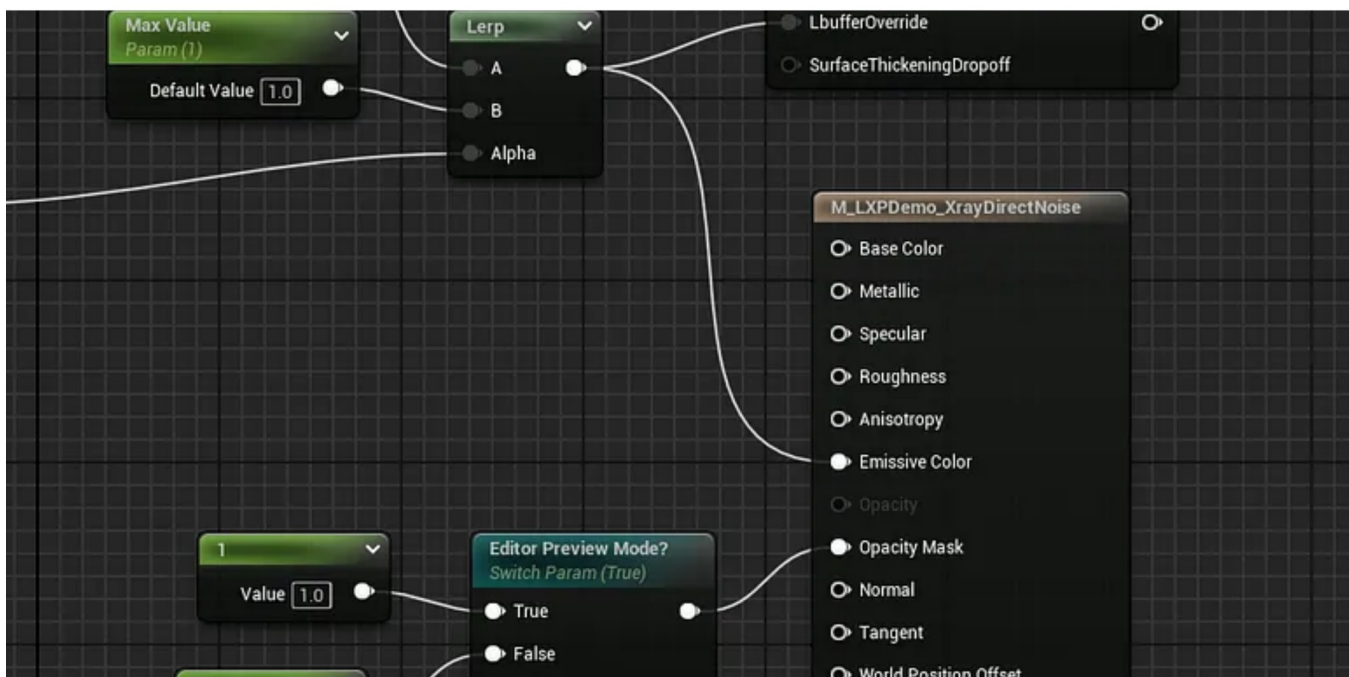
No responses yet



William Mishra-Manning

What are your thoughts?

More from William Mishra-Manning



William Mishra-Manning

Advanced Graphics Programming in Unreal, part 6

This is a seven-part article series.

Jun 25, 2025

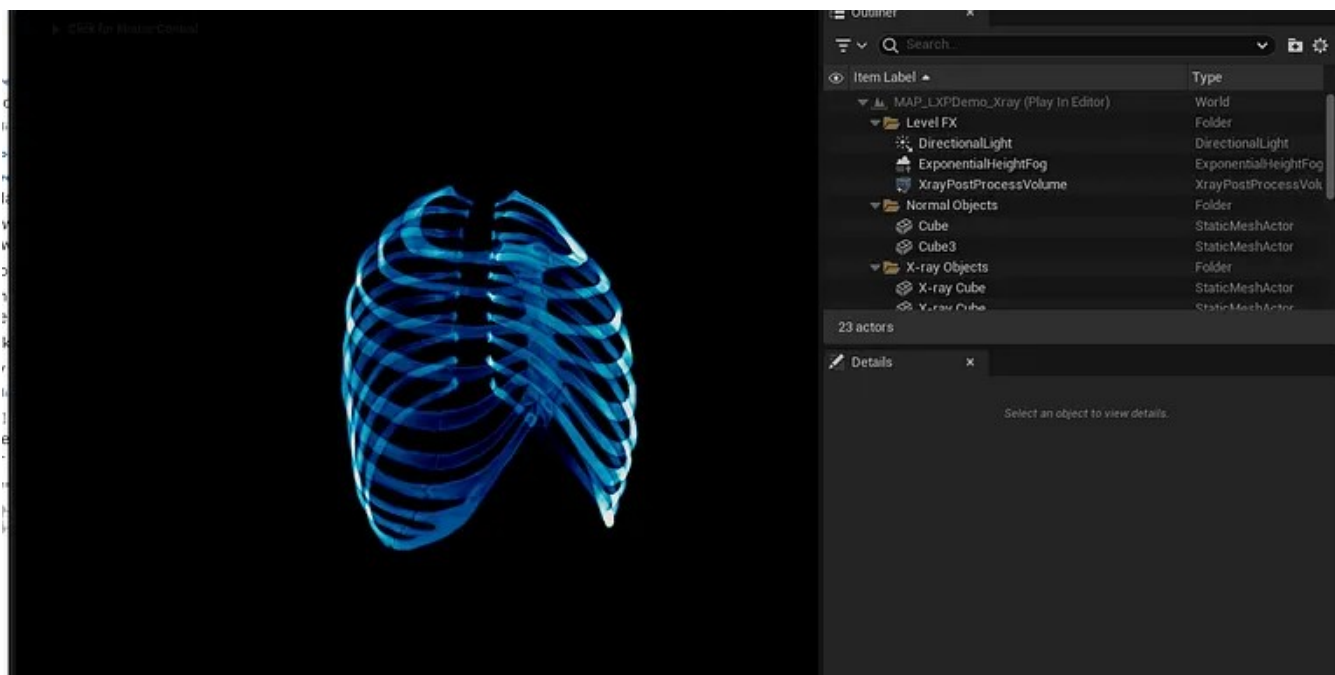


1



2



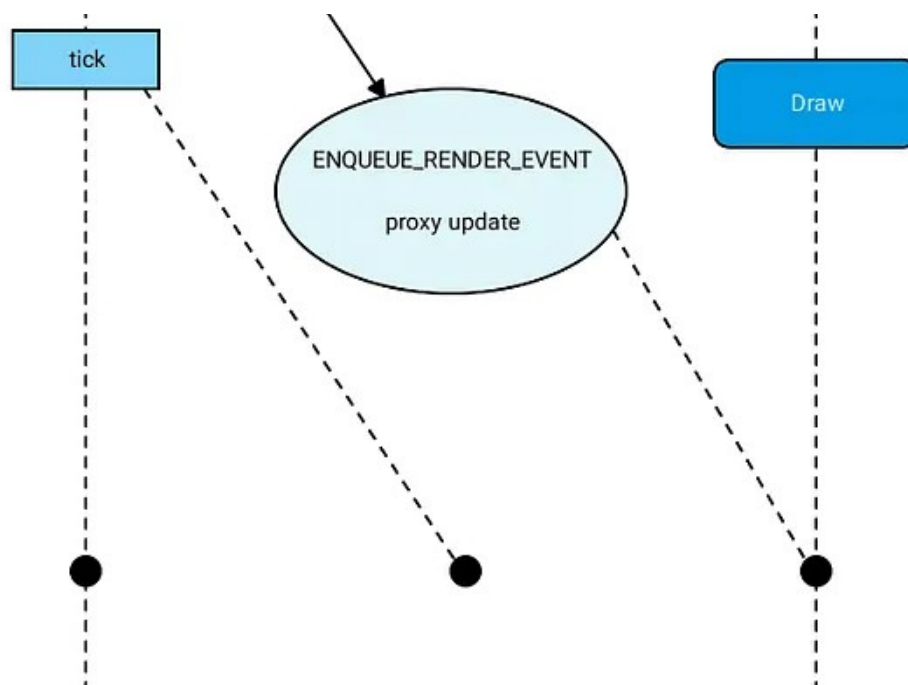


 William Mishra-Manning

Advanced Graphics Programming in Unreal, part 1

This is a seven-part article series.

Jun 24, 2025  31  1





William Mishra-Manning

Advanced Graphics Programming in Unreal, part 2

This is a seven-part article series.



William Mishra-Manning

Advanced Graphics Programming in Unreal, part 3

This is a seven-part article series.

Jun 24, 2025



6



See all from William Mishra-Manning

Recommended from Medium



Seulgie Han

C++ Move Semantics: From Copy to Efficient Resource Transfer

Source & Purpose: Based on Udacity C++ Nanodegree — Course3: Memory Management, this post is intended to help consolidate learning about...

Dec 13, 2025



 Kastuvpokharel

QNetworkInformation (C++) vs NetworkInformation (QML): performance, efficiency, security and what...

If you want to know what QNetworkInformation is read this short article first, as this will be the perquisite for this article...

Dec 15, 2025



Quick table		
Declaration	Change value?	Change address?
const int* p	✗	✓
int* const p	✓	✗
const int* const p	✗	✗
const int& r	✗	✗

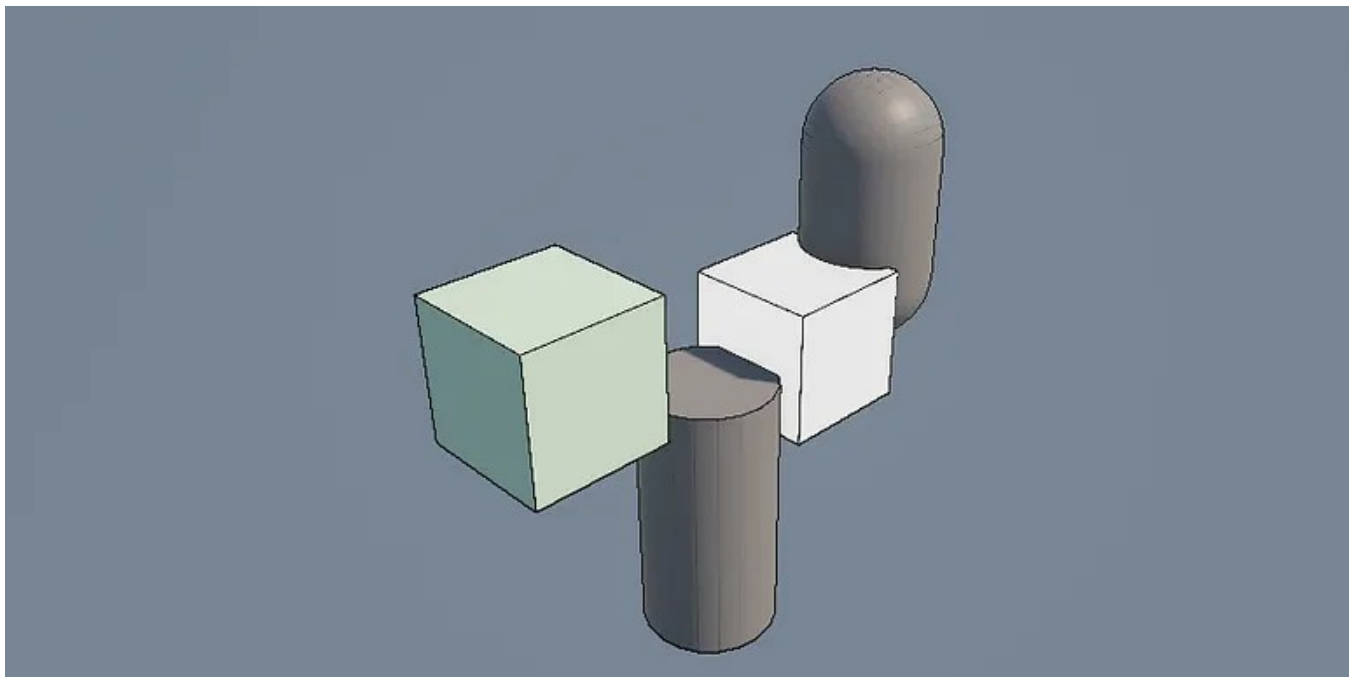
 Uday

Modern C++ : Part-1, Basics of C++

Basic Language Facilities

Dec 13, 2025





 madcritter

Creating a Full-Screen Outline Shader in Unity URP

Introduction

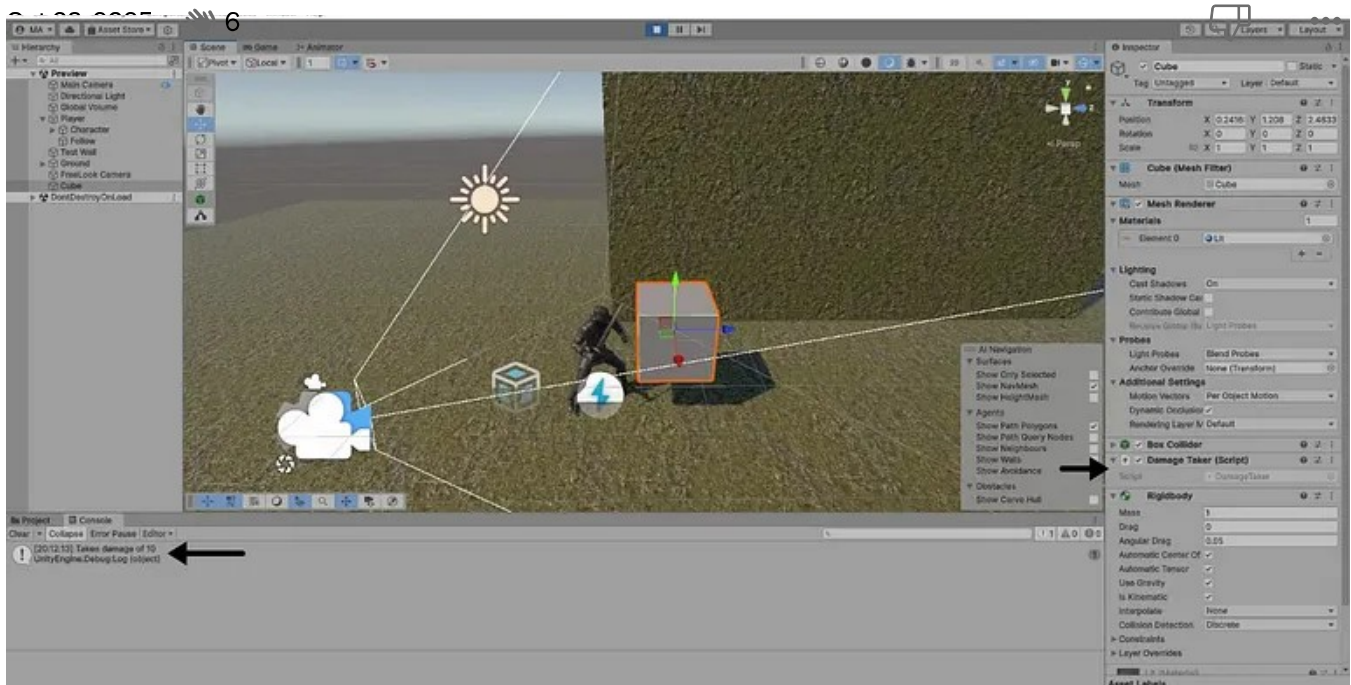
Oct 14, 2025  1



KM Kaloy Muñoz

Why CD Projekt Red Might Be the Last Prestige RPG Developer That Still Gets It

While the gaming industry shifts toward live-service models and diluted storytelling, CD Projekt Red continues to pursue ambitious...



Mahmoud Abaza

Building an RPG in Unity Part 3: Setting up camera and fighting system

In the previous chapter we explored how to add characters, animations and move them around into our game, but we have not given the camera...

Dec 16, 2025 9



See more recommendations